- `disable breakpoints`: the *breakpoints* will have no effect on your program.

4.8 Catching Exceptions

The command `catch exception-name` (or just `cat exception-name`) can be used to intercept an exception of type *exception-name* when there would otherwise be no handler for it.
To list all active catchpoints use `catch`.

4.9 Resuming Execution

There are two ways to resume execution of an application that is stopped in the debugger:

- `continue` [line-specification] (or `c`): resume program execution, at the address where your script last stopped; any breakpoints set at that address are bypassed. The optional argument line-specification allows you to specify a line number to set a one-time breakpoint which is deleted when that breakpoint is reached.
- `finish` [frame-number] (or `fin`): execute until the selected stack frame returns. If no frame number is given, the application will run until the currently selected frame returns. The currently selected frame starts out the most-recent frame or 0 if no frame positioning (e.g up, down or frame) has been performed. If a frame number is given it will run until the specified frame returns.

4.10 Editing

Two commands allow you to open code from the debugger into an editor:

- `edit [file:line]`: edit *file* using the editor specified by the EDITOR environment variable. A specific *line* can also be given.

4.11 Quitting

To exit the debugger, use the `quit` command (abbreviated `q`), or its alias `exit`.
A simple quit tries to terminate all threads in effect. Therefore your server will be stopped and you will have to start it again.

4.12 Settings

`byebug` has a few available options to tweak its behaviour:

- `set autoreload`: Reload source code when changed (default: true).
- `set autolist`: Execute `list` command on every breakpoint (default: true).
- `set listsize _n_`: Set number of source lines to list by default to *n* (default: 10)
- `set forcestep`: Make sure the `next` and `step` commands always move to a new line.

You can see the full list by using `help set`. Use `help set _subcommand_` to learn about a particular`set` command.
You can save these settings in an `.byebugrc` file in your home directory. The debugger reads these global settings when it starts. For example:

```
set forcestep
set listsize 25
```

5 Debugging Memory Leaks